CSS & Modern CSS: Zero to Hero

Each section gives the theory, a “which one should I choose?” guide, the code, and the live output of that code.

1 Selectors | 2 Cascade | 3 Box model | 4 Units & type | 5 Layout | 6 Positioning | 7 Responsive | 8 Modern CSS | 9 Motion | 10 Cheat sheet

1. What CSS is, and selectors

CSS (Cascading Style Sheets) controls how HTML looks. A rule has a **selector** (what to style) and **declarations** (`property: value;`). Add CSS three ways: inline `style=""`, a `<style>` block, or an external file with `<link rel="stylesheet">`.

> **Choose:** external file for real sites (shared, cached); `<style>` for one-off pages and demos; inline only for dynamic values set by JavaScript. Prefer **class** selectors for styling; use `#id` for anchors and JS hooks, and element selectors for site-wide defaults.

CODE

```
<style>
p { color: #444; }                    /* element */
.note { background: #fff3cd; padding: 6px; } /* class */
#main-title { color: teal; }          /* id */
a[href^="https"] { color: crimson; }  /* attribute */
li:nth-child(odd) { font-weight: bold; } /* pseudo-class */
li::marker { color: orange; }         /* pseudo-element */
</style>
<h3 id="main-title">Title (id)</h3>
<p>Plain paragraph (element)</p>
<p class="note">Highlighted note (class)</p>
<a href="https://example.com">A secure link</a>
<ul><li>One</li><li>Two</li><li>Three</li></ul>
```

OUTPUT

Title (id)

Plain paragraph (element)

Plain paragraph (element)

Highlighted note (class)

A secure link

- **One**
- Two
- **Three**

2. Cascade, specificity and inheritance

When rules conflict, the browser decides by: **importance** (`!important`), then **specificity** (inline > id > class/attribute/pseudo-class > element), then **source order** (later wins). Some properties (color, font) are **inherited** by children; others (margin, border) are not.

> **Choose:** keep specificity low and flat: mostly single classes. Fix conflicts with order or `@layer` (section 8), not `!important`, which is only for utility overrides.

CODE

```
<style>
p { color: blue; }
.warn { color: orange; }
p.warn { color: purple; }   /* more specific */
#big { color: green; }
.force { color: red !important; }
.parent { color: teal; }    /* inherited by children */
</style>
<p>Element rule: blue</p>
<p class="warn">p.warn beats .warn: purple</p>
<p id="big" class="warn">id beats class: green</p>
<p id="big" class="force">!important wins: red</p>
<div class="parent">Parent teal <span>child inherits teal</span></div>
```

OUTPUT

Element rule: blue

p.warn beats .warn: purple

id beats class: green

!important wins: red

Parent teal child inherits teal

3. The box model

Every element is a box: **content**, **padding** (inside), **border**, **margin** (outside). By default `width` sets only the content, so padding and border add to it. `box-sizing: border-box` makes width include them, which is far easier to reason about.

> **Choose:** always set `*,*::before,*::after{box-sizing:border-box}` at the top of a project. Use **margin** to space elements apart, **padding** to space content from its own edge, and `gap` inside flex/grid.

CODE

```
<style>
.box { width: 200px; padding: 20px; border: 5px solid teal; margin: 10px 0; background: #cff; }
.bb { box-sizing: border-box; }
</style>
<div class="box">content-box: 250px wide in total</div>
<div class="box bb">border-box: 200px wide in total</div>
```

OUTPUT

content-box: 250px wide in total

border-box: 200px wide in total

4. Colours, units and typography

Units: **px** (fixed), **rem** (relative to root font size), **em** (relative to the element's font size), **%** (of the parent), **vw/vh** (of the viewport). Colours: names, hex, `rgb()`, `hsl()`, and modern `oklch()`. `clamp(min, ideal, max)` makes fluid sizes with no media query.

> **Choose: rem** for font sizes and spacing (respects user settings); **px** for hairline borders; **%** or **fr** for widths; **vw** or **clamp()** for fluid headings; **hsl/oklch** when you need to tweak lightness.

CODE

```
<style>
.t { font: clamp(1rem, 4vw, 1.8rem)/1.4 Georgia, serif; color: hsl(170 80% 25%); }
.s { font-size: 1.25rem; padding: .5em 1em; background: oklch(90% .08 250); width: 50%; border-ra
</style>
<p class="t">Fluid text sized with clamp()</p>
<div class="s">1.25rem text, 50% width</div>
```

OUTPUT

Fluid text sized with clamp()

1.25rem text, 50% width

5. Layout: normal flow, Flexbox, Grid

Flexbox arranges items along *one axis* (a row or a column) and distributes space. **Grid** arranges items in *rows and columns at once*. Normal flow (block and inline) is already good for text.

> **Choose: Flexbox** for navbars, toolbars, centring, button groups, cards in a row. **Grid** for page layouts, galleries, dashboards, anything with rows *and* columns. Combine them: grid for the page, flex inside components. Avoid floats and table layouts.

CODE

```
<style>
.row { display: flex; gap: 8px; justify-content: space-between; align-items: center; }
.grid { display: grid; gap: 8px; margin-top: 12px;
        grid-template-columns: repeat(auto-fit, minmax(90px, 1fr)); }
```

```
.row div, .grid div { background: #bde; padding: 12px; border-radius: 6px; text-align: center; }
</style>
<div class="row"><div>Logo</div><div>Menu</div><div>Login</div></div>
<div class="grid"><div>1</div><div>2</div><div>3</div><div>4</div><div>5</div><div>6</div></div>
```

OUTPUT



6. Positioning

`static` is the default. `relative` keeps the element in flow and becomes the anchor for children. `absolute` removes it from flow and places it relative to the nearest positioned ancestor. `fixed` sticks to the viewport. `sticky` scrolls normally, then sticks at a threshold. `z-index` sets stacking.

> **Choose: absolute** inside a **relative** parent for badges and overlays; **sticky** for headers and table headings; **fixed** for chat buttons and cookie bars. Do not position whole layouts: use flex or grid.

CODE

```
<style>
.wrap { position: relative; height: 90px; background: #eee; padding: 10px; }
.badge { position: absolute; top: 8px; right: 8px; background: crimson; color: #fff; padding: 2px
</style>
<div class="wrap">A card with a badge in the corner <span class="badge">New</span></div>
```

OUTPUT



7. Responsive design

Start **mobile-first** with the viewport meta tag. **Media queries** respond to the *viewport*; **container queries** respond to the *parent's* size, so a component adapts wherever it is placed.

> **Choose: fluid techniques first** (`clamp`, `auto-fit` grids, flex-wrap) so you need fewer breakpoints; **media queries** for page-level changes (nav to hamburger); **container queries** for reusable components; `prefers-color-scheme` and `prefers-reduced-motion` for user settings.

CODE

```
<style>
.cw { container-type: inline-size; resize: horizontal; overflow: auto; border: 2px dashed #888; p
.card { display: grid; gap: 8px; }
.pic { background: teal; height: 60px; border-radius: 6px; }
@container (min-width: 380px) { .card { grid-template-columns: 90px 1fr; } }
/* page-level equivalent: @media (min-width: 700px) { ... } */
</style>
<p>Drag the bottom-right corner to resize:</p>
<div class="cw"><div class="card"><div class="pic"></div><div>Narrow: stacked. Wide: image beside
```

OUTPUT

Drag the bottom-right corner to resize:

Narrow: stacked. Wide: image beside text.

8. Modern CSS essentials

Custom properties (`--brand`) hold reusable values and can change at runtime. **Nesting** lets you write child rules inside a rule. **:has()** selects a parent based on its children. `:is()` and `:where()` shorten selector lists. `color-mix()` blends colours. `aspect-ratio` keeps proportions. `@layer` sets cascade order explicitly. **Logical properties** (`margin-inline`, `padding-block`) adapt to writing direction.

> **Choose:** variables for themes and spacing scales; nesting for component-scoped styles;

> `:has()` instead of JavaScript for "style the form if a field is checked or invalid"; `@layer` to beat specificity wars when mixing resets, libraries and your own code. Check browser support on MDN before relying on newer features.

CODE

```
<style>
:host { --brand: #0f766e; --pad: 12px; }
.card {
  padding: var(--pad);
  border: 2px solid var(--brand);
  background: color-mix(in srgb, var(--brand) 15%, white);
  aspect-ratio: 3 / 1;
  margin-inline: auto;
  & h4 { margin: 0; color: var(--brand); }      /* nesting */
  &:hover { background: color-mix(in srgb, var(--brand) 30%, white); }
}
label:has(input:checked) { background: gold; padding: 4px 8px; border-radius: 6px; }
</style>
<div class="card"><h4>Variables, nesting, color-mix</h4>Hover me.</div>
<p><label><input type="checkbox"> Tick me: :has() styles the label</label></p>
```

OUTPUT

Variables, nesting, color-mix

Hover me.

☐ Tick me: :has() styles the label

9. Transitions and animations

A **transition** smoothly changes a property between two states (like hover). An **animation** uses `@keyframes` to run multi-step or looping motion. Animate `transform` and `opacity`: they are

cheap for the browser.

> **Choose: transition** for interaction feedback (hover, focus, open/close); **@keyframes** for loaders and attention effects; JavaScript only when motion depends on logic. Always respect `prefers-reduced-motion`.

CODE

```
<style>
.btn { padding: 10px 18px; border: 0; border-radius: 8px; background: teal; color: #fff; cursor:
       transition: transform .2s, background .2s; }
.btn:hover { transform: translateY(-3px); background: #0a7a5a; }
.dot { width: 20px; height: 20px; border-radius: 50%; background: crimson; margin-top: 14px;
       animation: slide 1s ease-in-out infinite alternate; }
@keyframes slide { to { transform: translateX(160px); } }
@media (prefers-reduced-motion: reduce) { .dot { animation: none; } }
</style>
<button class="btn">Hover me (transition)</button>
<div class="dot"></div>
```

OUTPUT

Hover me (transition)

10. Cheat sheet: what to choose when

Scenario	Choose
Centre something	Flex: `display:flex; justify-content:center; align-items:center` or grid: `place-items:center`
Navbar or button row	Flexbox with `gap`
Page layout, galleries	Grid, with `auto-fit` and `minmax()`
Badge on a card	Relative parent plus absolute child
Header that stays visible	Sticky
Text size on all screens	rem units plus `clamp()`
Re-usable component that	Container queries

Reusable component that adapts	Container queries
Whole-page layout changes	Media queries, mobile-first
Theme colours or spacing scale	Custom properties
Style a parent from its child state	`:has()`
Specificity fights	Lower specificity, then `@layer`; avoid `!important`
Hover and focus feedback	Transition on transform and opacity
Loading spinner or looping motion	@keyframes animation

Next steps: practise by rebuilding a site you like, then learn CSS Grid areas, `@layer` in depth, scroll-driven animations and view transitions. MDN Web Docs is the best reference.